VIRTUAL MACHINE FRAMEWORK PROJECT

Organisational Structure and Role Definitions

Excruciating Detail Edition

15 Roles | One Human | March 2026

Overview

This document defines the complete organisational structure for the Virtual Machine Framework project. Every role is specified with full accountability: what it receives, what it produces, who it reports to, what authority it holds, and what constraints it operates under.

The system is designed to run as an AI multi-agent organisation inside Claude Code. Each role maps to one or more agent instances. The human (Marko) operates as analyst and ultimate authority. Everything else is an agent.

Fifteen roles. One human. The question was whether this is possible. The answer is being built.

Role Summary

Abbrev	Role	Core function
PM	Project Manager	*The external face. The internal hub. The one who holds the full picture.*
SD	Software Designer	*Turns analysis into architecture. Owns the contracts between everything.*
PPM	Program Project Manager	*Breaks the big stack into workable pieces. Assigns. Tracks. Unblocks.*
SPM	Sub System Manager	*Deeper than the PPM. Owns the technical domain. Runs the coders.*
TM	Subsystem Tool Maker	*Builds the tools the coders in their subsystem actually need.*
MTM	Master Tool Maker	*Finds what gets built six times and makes it once. Properly.*
CR	Code Reviewer	*Not the coder. Never the coder. Finds what the coder cannot see.*
SEC	Security	*Standards first. Not a gate at the end. A constraint from the beginning.*
UX	UI/UX Designer	*As close to human as possible. No exceptions.*
DOC	Documenters	*Every role produces something. Someone has to capture it properly.*
TEST	Testers	*Real coverage. If the tool does not exist to test it, the tool gets built.*
INT	Integrators	*Does not build subsystems. Composes them. Where the gaps show up.*
QA	Quality Manager	*Measurable. Defined. Pass or fail. No bullshit.*
CMVC	Configuration Management and Version Control	*Code moves forward when it is ready. Releases happen when there is enough of it.*
INFRA	Infrastructure	*Horizontal. Serves everyone simultaneously. Belongs to no one exclusively.*

PM Project Manager

The external face. The internal hub. The one who holds the full picture.

Mission

The Project Manager owns all communication with the client and all coordination downward through the organisation. He is the single point of accountability for the project from the client's perspective and the single source of organisational truth from the team's perspective.

Reports to	Client
Manages	PPM, Software Designer, Infrastructure, CMVC, Security, Quality
Communicates with	All roles via daily reports and escalation paths

Inputs

- Client requirements, change requests, and feedback
- Daily status reports from PPM, Designer, Security, Quality, CMVC, and Infrastructure
- Escalations from any role in the org
- Risk and issue logs

Outputs

- Project plan and milestone schedule
- Client-facing status reports and presentations
- Decisions on scope, priority, and resource allocation
- Escalation responses and unblocking actions
- Organisational role assignments and authority definitions

Responsibilities

- Maintain complete situational awareness across all roles and subsystems
- Know the strengths and weaknesses of every agent in the system
- Manage client expectations proactively, not reactively
- Receive and review daily reports from all direct reports
- Make final decisions on priority conflicts between subsystems
- Approve scope changes before they reach the Designer or PPM
- Monitor overall project health and intervene before problems compound

Authority

Full authority over project scope, priority, and resource allocation. Final decision on all conflicts not resolved at lower levels.

Constraints

Cannot override security standards or quality gates. Must document all client decisions.

Agent implementation notes

The PM agent reads the full project state daily. Its system prompt includes complete team capability profiles. It is the only agent authorised to communicate with the human client layer.

SD Software Designer

Turns analysis into architecture. Owns the contracts between everything.

Mission

The Software Designer takes the analyst output produced by the human and translates it into formal interface and system design documents. He owns the communication contracts between subsystems, the data exchange specifications, and the structural design of the entire system. He does not produce analysis. He consumes it.

Reports to	PM
Manages	No direct reports
Communicates with	PM (upward), PPM and SPM (design artifacts downward), Security (interface compliance), Documenters (spec handoff)

Inputs

- Analysis documents produced by the human (Marko)
- Client requirements passed down from PM
- Security standards from the Security role
- Feedback from PPM and SPM on design feasibility

Outputs

- Interface specification documents consumed by PPM and SPM
- Subsystem communication maps defining which subsystem talks to which
- Data exchange definitions including format, protocol, and validation rules
- Design decision log
- Updated specs when requirements change

Responsibilities

- Define all interfaces between subsystems with precision
- Specify data exchange formats, protocols, and validation requirements
- Ensure the design is implementable by the coding crew
- Flag design conflicts or ambiguities before they reach the coders
- Update specifications when requirements change and notify PPM and SPM
- Coordinate with Security to ensure interface design meets security standards

- Hand off complete spec packages to Documenters

Authority

Full authority over interface design decisions. Can reject implementation proposals that violate design contracts.

Constraints

Cannot change requirements unilaterally. Must consume human analysis as the source of truth. All design decisions must be documented.

Agent implementation notes

The Designer agent receives the human's analysis as its primary input document. It outputs structured spec files that become the working reference for PPM and SPM agents.

PPM Program Project Manager

Breaks the big stack into workable pieces. Assigns. Tracks. Unblocks.

Mission

The PPM takes the design output from the Software Designer and breaks it into concrete implementation tasks. He assigns those tasks to the coding crew through the SPM layer and tracks progress across all active work. The coding crew size is dynamic and scales to the current workload.

Reports to	PM
Manages	SPMs, dynamic coding crew allocation
Communicates with	SD (design inputs), SPMs (task distribution), PM (status upward)

Inputs

- Interface and system design documents from the Software Designer
- Status reports from SPMs
- Priority decisions from PM
- Capacity information from SPMs on coder availability

Outputs

- Task breakdown documents with clear scope, dependencies, and acceptance criteria
- Task assignments to SPMs with priority and deadline
- Daily progress reports to PM
- Escalations to PM when blocked
- Dynamic coder allocation decisions

Responsibilities

- Decompose design documents into implementable tasks with no ambiguity
- Maintain a prioritised backlog of all pending work
- Assign tasks to the appropriate SPM based on subsystem ownership
- Track completion status across all active tasks
- Resolve dependency conflicts between subsystems
- Scale coder allocation up or down as workload demands

- Report accurate progress to PM daily

Authority

Authority over task assignment, prioritisation, and coding crew allocation. Can reassign tasks between SPMs.

Constraints

Cannot override design specifications. Cannot approve completed work. That is the Code Reviewer and Quality Manager's function.

Agent implementation notes

The PPM agent maintains a shared task state file. SPM agents read from this file and write completion status back to it. The PPM agent never writes code.

SPM Sub System Manager

Deeper than the PPM. Owns the technical domain. Runs the coders.

Mission

The SPM sits between the PPM and the coding crew with deeper technical knowledge of their specific subsystem than the PPM holds. Each SPM owns one subsystem domain. If the project has many subsystems there may be multiple SPMs, each accountable for their domain. Coders report to their SPM, not directly to the PPM.

Reports to	PPM
Manages	Coders assigned to their subsystem, Subsystem Tool Maker
Communicates with	PPM (task intake), Coders (task distribution), Code Reviewers (review coordination), Integrators (handoff)

Inputs

- Task assignments from PPM with scope and acceptance criteria
- Interface specifications from Software Designer relevant to their subsystem
- Code review results from Code Reviewers
- Tool availability from their Subsystem Tool Maker

Outputs

- Task assignments to individual coders with technical context
- Status reports to PPM
- Technical decisions within their subsystem scope
- Subsystem handoff packages to Integrators
- Tool requests to their Subsystem Tool Maker

Responsibilities

- Maintain deep technical understanding of their assigned subsystem
- Translate PPM task assignments into technically precise coder instructions
- Make technical decisions within subsystem scope without escalating everything
- Coordinate code reviews for their subsystem's output
- Ensure coders have the tools they need before work begins
- Manage dependencies between their subsystem and others via PPM

- Own the quality of the subsystem output before it reaches formal review

Authority

Full technical authority within their subsystem. Can make implementation decisions without PPM approval. Cannot override interface specifications.

Constraints

Must operate within the interface contracts defined by the Software Designer. Cannot approve their own subsystem's output for release.

Agent implementation notes

Multiple SPM agents can run simultaneously, each scoped to a subsystem context. They share a read-only view of the full design but write only to their subsystem's state.

TM Subsystem Tool Maker

Builds the tools the coders in their subsystem actually need.

Mission

Each subsystem has its own Tool Maker. This role exists to build specific, contextual tooling for the coders working within that subsystem. Generic tools are not the Tool Maker's output. Purpose-built tools that solve the exact problems that subsystem's coders face are.

Reports to	SPM
Manages	No direct reports
Communicates with	SPM (tool requirements), Coders (tool delivery), Master Tool Maker (escalation for cross-cutting tools)

Inputs

- Tool requirements from the SPM based on coder friction points
- Understanding of the subsystem's specific implementation context
- Shared tooling from the Master Tool Maker

Outputs

- Purpose-built tools scoped to their subsystem's needs
- Tool documentation handed to Documenters
- Escalation requests to Master Tool Maker for tools needed across subsystems

Responsibilities

- Observe what the coders in their subsystem struggle with and build solutions
- Build tools before the coders need them when possible
- Maintain and update existing tools as the subsystem evolves
- Document every tool so the SPM and coders can use them without friction
- Identify when a tool they are building has cross-subsystem value and escalate to Master Tool Maker
- Never build what the Master Tool Maker already provides

Authority

Full authority to build any tooling within their subsystem scope.

Constraints

Tools must be documented before delivery. Cannot deploy tools that affect other subsystems without Master Tool Maker coordination.

Agent implementation notes

Subsystem Tool Maker agents operate in the same context as their subsystem's coders. They produce executable tools, scripts, and utilities as their primary output.

MTM Master Tool Maker

Finds what gets built six times and makes it once. Properly.

Mission

The Master Tool Maker sits above the Subsystem Tool Makers and owns the shared tooling layer used across the entire project. When something is being built repeatedly in multiple subsystems, it belongs here. He also owns the review tooling used by Code Reviewers and the test tooling used by Testers if those tools do not already exist.

Reports to	PPM
Manages	No direct reports. Coordinates with all Subsystem Tool Makers.
Communicates with	All Subsystem Tool Makers (coordination), PPM (scope alignment), Code Reviewers and Testers (shared tooling delivery)

Inputs

- Escalations from Subsystem Tool Makers identifying cross-cutting tool needs
- Tool requests from Code Reviewers and Testers
- Patterns identified across subsystems indicating shared tooling opportunities

Outputs

- Shared tools available to all subsystems
- Code review toolset used by all Code Reviewers
- Test tooling used by all Testers where not already available
- Tool registry documenting all shared tools and their interfaces
- Deprecation notices when shared tools are superseded

Responsibilities

- Monitor what Subsystem Tool Makers are building for duplication
- Abstract repeated solutions into shared tools that all subsystems can import
- Build and maintain the code review toolset as a shared standard
- Build test tooling when it does not exist and the Testers need it
- Maintain the master tool registry so every role knows what exists
- Version shared tools and communicate breaking changes

Authority

Authority over the shared tooling layer. Can standardise a subsystem tool into a shared tool with PPM alignment.

Constraints

Cannot build tools that violate security standards. All shared tools must be documented and versioned.

Agent implementation notes

The Master Tool Maker agent maintains a tool registry file readable by all other agents. It is the authoritative source for what shared tooling exists.

CR Code Reviewer

Not the coder. Never the coder. Finds what the coder cannot see.

Mission

Code Reviewers are structurally separated from the coders who produced the code. This is not a courtesy. It is a design constraint. The reviewer who built the thing cannot be genuinely adversarial toward it. Code Reviewers use a standardised shared toolset built and maintained by the Master Tool Maker and can requisition new tools when needed.

Reports to	SPM (for subsystem reviews), PPM (for cross-subsystem work)
Manages	No direct reports
Communicates with	Coders (review feedback), SPM (review status), Master Tool Maker (tool requests), Quality (standards alignment)

Inputs

- Code submitted for review by Coders via SPM
- Interface specifications from Software Designer as the review standard
- Code review toolset from Master Tool Maker
- Quality standards from Quality Manager
- Security standards from Security

Outputs

- Review reports with specific, actionable findings
- Approval or rejection decision with documented rationale
- Tool requests to Master Tool Maker when review gaps exist
- Patterns of recurring issues reported to SPM and PPM

Responsibilities

- Review all code against interface specifications and quality standards
- Apply the shared review toolset consistently across all reviews
- Produce specific, actionable review findings with no ambiguity
- Never approve code that does not meet the defined standards
- Identify recurring problem patterns and escalate to SPM
- Requisition new review tools from Master Tool Maker when coverage gaps exist

- Complete reviews within defined turnaround time

Authority

Full authority to reject code. Rejection cannot be overridden by a Coder or SPM. Can only be resolved by fixing the code.

Constraints

Must use the shared toolset for consistency. Cannot review code they wrote. All review decisions must be documented.

Agent implementation notes

Code Reviewer agents receive code packages and spec documents. They never share context with the Coder agent that produced the same code.

SEC Security

Standards first. Not a gate at the end. A constraint from the beginning.

Mission

Security defines the standards the entire system is built against. These standards are not retrofitted after the fact. They are defined before design, consumed by the Designer when specifying interfaces, applied by SPMs when directing coders, and enforced by Code Reviewers and Quality at review time. Security is a constraint that runs through the entire pipeline.

Reports to	PM
Manages	No direct reports
Communicates with	SD (interface compliance), SPM (implementation standards), Code Reviewers (enforcement), Quality (gate alignment), Integrators (integration security), CMVC (release security)

Inputs

- Industry security standards applicable to the project
- Client security requirements from PM
- Interface designs from Software Designer for security review
- Code review results flagging security findings
- Integration reports from Integrators

Outputs

- Security standards document consumed by all roles
- Interface security review findings to Software Designer
- Security acceptance criteria used by Code Reviewers and Quality
- Security sign-off for CMVC release gate
- Security incident reports to PM

Responsibilities

- Define security standards before design begins
- Review interface specifications for security compliance
- Provide clear, measurable security acceptance criteria
- Coordinate with Code Reviewers on security-specific review coverage

- Participate in integration review with Integrators and Quality
- Provide security sign-off as part of the CMVC release gate
- Update standards when the threat landscape or requirements change

Authority

Authority to block release on security grounds. Security findings cannot be waived by PM without documented client acceptance.

Constraints

Security standards must be defined in measurable terms. Cannot block indefinitely without documented justification.

Agent implementation notes

Security agent outputs a standards document that is injected into the context of Designer, Code Reviewer, and Quality agents as a read-only reference.

UX UI/UX Designer

As close to human as possible. No exceptions.

Mission

The UI/UX Designer owns all user-facing surfaces. The design standard is clear: as close to human and intuitive as possible. Not minimal for its own sake. Not technically impressive at the cost of usability. The interface should feel like it was made for the person using it, not for the engineer who built it.

Reports to	SD
Manages	No direct reports
Communicates with	SD (design alignment), SPM for UI subsystem (implementation handoff), Testers (usability review), Code Reviewers (UI code standards)

Inputs

- Analysis documents from human (Marko) for user context
- System design from Software Designer
- User feedback and usability findings from Testers
- Technical constraints from the relevant SPM

Outputs

- UI/UX design specifications consumed by the UI subsystem SPM and coders
- Component definitions and interaction patterns
- Design tokens and visual standards
- Usability acceptance criteria for Testers

Responsibilities

- Design all user-facing interfaces to the highest standard of intuitiveness
- Produce specifications clear enough for coders to implement without guesswork
- Define usability acceptance criteria before implementation begins
- Review implemented interfaces against specification
- Iterate based on usability findings from Testers
- Maintain design consistency across all surfaces

Authority

Authority over all UI/UX design decisions. Can reject implementations that do not meet the usability standard.

Constraints

Must operate within technical constraints defined by the relevant SPM. Cannot change underlying system architecture for aesthetic reasons.

Agent implementation notes

The UX agent produces design spec documents consumed by the UI subsystem. It also receives usability test results and iterates specifications accordingly.

DOC Documenters

Every role produces something. Someone has to capture it properly.

Mission

Documenters capture the output of every other role in a form that is accurate, usable, and does not require the next reader to reverse engineer what the previous person was thinking. Documentation is not a post-project activity. It is concurrent with the work.

Reports to	PM
Manages	No direct reports
Communicates with	All roles (documentation intake), CMVC (release documentation), PM (documentation status)

Inputs

- Design specifications from Software Designer
- Tool documentation from Tool Makers
- API and interface documentation from Coders and SPMs
- Process decisions from PM and PPM
- Security standards from Security
- Test plans and results from Testers

Outputs

- Technical documentation for all subsystems
- API and interface reference documentation
- Tool usage guides
- Release documentation packages for CMVC
- User-facing documentation where required
- Internal process documentation

Responsibilities

- Capture documentation concurrently with the work, not after it
- Ensure documentation matches the actual implementation, not the intended one
- Maintain a documentation registry so every role knows what exists
- Flag documentation gaps to the relevant role

- Produce release documentation packages for CMVC
- Update documentation when the system changes

Authority

Authority to request documentation inputs from any role. Can flag undocumented work to PM.

Constraints

Cannot approve work as complete. That is Quality's function. Must document what was built, not what was planned.

Agent implementation notes

Documenter agents receive structured outputs from other agents and produce formatted documentation files stored in a shared documentation directory.

TEST Testers

Real coverage. If the tool does not exist to test it, the tool gets built.

Mission

Testers verify that what was built does what it is supposed to do, under the conditions it is supposed to do it in. If the tooling required to test something does not exist, a request goes to the Master Tool Maker to build it. No coverage gaps are accepted because the harness is missing.

Reports to	PPM
Manages	No direct reports
Communicates with	Master Tool Maker (test tool requests), SPM (test scope alignment), Integrators (integration testing), Quality (results handoff), Security (security test coordination)

Inputs

- Acceptance criteria from PPM and SPM
- Interface specifications from Software Designer
- Security acceptance criteria from Security
- Code and subsystem packages from SPMs
- Test tooling from Master Tool Maker
- Integration packages from Integrators

Outputs

- Test plans with coverage definition
- Test results with pass, fail, and coverage metrics
- Defect reports with reproduction steps to SPM
- Integration test results to Integrators and Quality
- Test tool requests to Master Tool Maker
- Usability findings to UI/UX Designer

Responsibilities

- Define test plans before testing begins
- Achieve defined coverage targets. No gaps accepted.

- Produce reproducible defect reports with full context
- Coordinate with Integrators on integration-level testing
- Request new test tooling from Master Tool Maker when coverage gaps exist
- Provide usability findings to UI/UX Designer
- Confirm defect fixes before marking resolved

Authority

Authority to block progression on failed tests. A failing test cannot be waived without PM and Quality sign-off.

Constraints

Must define coverage targets before testing. Cannot approve their own test plans. Plans must be reviewed by Quality.

Agent implementation notes

Tester agents receive code packages and spec documents and produce structured test result files. They can spawn tool-request tasks to the Master Tool Maker agent.

INT Integrators

Does not build subsystems. Composes them. Where the gaps show up.

Mission

Integrators take validated subsystem outputs and assemble them into the final system. They do not build. They compose. Integration is where the gaps between subsystems become visible and where the triad of Testers, Quality, and Security converges on the assembled result.

Reports to	PPM
Manages	No direct reports
Communicates with	SPMs (subsystem handoff), Testers (integration testing), Quality (integration quality gate), Security (integration security review), CMVC (release package handoff)

Inputs

- Validated subsystem packages from SPMs with Code Reviewer approval
- Interface specifications from Software Designer as the integration contract
- Integration test plans from Testers
- Security integration requirements from Security

Outputs

- Integrated system builds
- Integration test request packages to Testers
- Integration issues reported to relevant SPM
- Release candidate packages to CMVC
- Integration status reports to PPM

Responsibilities

- Verify subsystem packages meet interface contracts before integrating
- Assemble subsystems in dependency order
- Coordinate integration testing with Testers
- Identify and report integration gaps back to the relevant SPM
- Work with Quality and Security on the integration quality gate
- Deliver release candidate packages to CMVC when integration passes

Authority

Authority to reject subsystem packages that do not meet interface contracts. Can request rework before integration.

Constraints

Cannot override interface specifications. Cannot approve their own integration work. That is Quality and Security's function.

Agent implementation notes

Integrator agents maintain an integration state file tracking which subsystems have been validated and assembled. They coordinate with Tester, Quality, and Security agents at each integration stage.

QA Quality Manager

Measurable. Defined. Pass or fail. No bullshit.

Mission

Quality defines what done means in measurable terms before work begins and enforces those standards at every gate in the pipeline. Quality works with Testers, Security, and Integrators at integration time. Quality does not accept undefined standards, approximate compliance, or exceptions without documented rationale.

Reports to	PM
Manages	No direct reports
Communicates with	PM (standards approval), Testers (test quality alignment), Security (joint gate), Integrators (integration gate), Code Reviewers (review standard alignment), CMVC (release quality sign-off)

Inputs

- Project quality requirements from PM and client
- Security standards from Security
- Test results from Testers
- Code review results from Code Reviewers
- Integration results from Integrators

Outputs

- Quality standards document with measurable acceptance criteria
- Quality gate results at each pipeline stage
- Quality sign-off for CMVC release gate
- Quality findings reports to PM
- Defect classification and severity assignments

Responsibilities

- Define measurable quality standards before any work begins
- Apply those standards at every gate without exception
- Coordinate the integration quality gate with Testers and Security
- Produce clear pass or fail decisions with documented rationale

- Escalate quality failures to PM when they cannot be resolved at lower levels
- Provide quality sign-off as part of the CMVC release gate
- Track quality metrics across the project and report trends to PM

Authority

Authority to block any progression on quality grounds. Quality failures cannot be waived by PPM or SPM. PM sign-off required for any exception.

Constraints

Standards must be defined in measurable terms before enforcement. Cannot measure against undefined standards.

Agent implementation notes

Quality agent receives structured outputs from Testers, Code Reviewers, and Integrators. It applies defined standards and produces binary pass/fail outputs with documented rationale.

CMVC Configuration Management and Version Control

Code moves forward when it is ready. Releases happen when there is enough of it.

Mission

CMVC owns the configuration control and version management layer for the entire project. Code does not move forward until it meets the bar. Releases happen when there is sufficient validated, integrated, quality-approved work to warrant one. CMVC also owns the release process end to end.

Reports to	PM
Manages	No direct reports
Communicates with	All roles (version and branch standards), Integrators (release package intake), Quality and Security (release gate), PM (release decisions), Documenters (release documentation)

Inputs

- Release candidate packages from Integrators
- Quality sign-off from Quality Manager
- Security sign-off from Security
- Release documentation from Documenters
- Release decision from PM

Outputs

- Branching strategy and versioning standards for all roles
- Configuration baseline definitions
- Release packages with full provenance
- Release notes
- Version history and change log

Responsibilities

- Define and enforce branching strategy and versioning standards across all roles
- Maintain configuration baselines at each project milestone
- Receive release candidate packages from Integrators
- Verify Quality and Security sign-off before promoting to release

- Execute the release process when PM approves
- Maintain complete release history with full traceability
- Produce release notes in collaboration with Documenters

Authority

Authority to block releases that do not have Quality and Security sign-off. Authority over branching standards and version control policy across all roles.

Constraints

Cannot release without Quality and Security sign-off. Cannot override PM release decision.

Agent implementation notes

CMVC agent maintains the version registry and release state file. It is the final gate before anything exits the system. It reads sign-off outputs from Quality and Security agents before promoting.

INFRA Infrastructure

Horizontal. Serves everyone simultaneously. Belongs to no one exclusively.

Mission

Infrastructure owns the build pipelines, environments, containerisation, and deployment infrastructure that every other role depends on. It does not build features. It builds and maintains the platform that the entire organisation runs on. Infrastructure serves Coders, Testers, Integrators, CMVC, and Security simultaneously and reports to the PM.

Reports to	PM
Manages	No direct reports
Communicates with	All roles (infrastructure provisioning and support), PPM (capacity coordination), Security (infrastructure security compliance), CMVC (build and deploy pipeline)

Inputs

- Infrastructure requirements from PPM based on project needs
- Security requirements from Security for infrastructure compliance
- Environment requests from SPMs and Testers
- Pipeline requirements from CMVC

Outputs

- CI/CD pipeline configuration and maintenance
- Development, test, and integration environments
- Infrastructure documentation to Documenters
- Capacity and environment status reports to PM
- Infrastructure security compliance reports to Security

Responsibilities

- Build and maintain the CI/CD pipeline used by all roles
- Provision and maintain development, test, and integration environments
- Ensure infrastructure meets security standards defined by Security
- Respond to environment requests from SPMs and Testers
- Coordinate with CMVC on build and deploy pipeline requirements

- Monitor infrastructure health and report issues proactively to PM
- Scale infrastructure as project demand grows

Authority

Full authority over infrastructure design and implementation. Can define infrastructure standards that all roles must comply with.

Constraints

Must meet security standards. Infrastructure changes that affect the release pipeline require CMVC coordination.

Agent implementation notes

Infrastructure agent maintains environment state and pipeline configuration. Other agents can query infrastructure status but cannot modify environments directly.